

Guía de Laboratorio

Observabilidad de Microservicios con Grafana, Prometheus, Loki y OpenTelemetry

Asignatura: Arquitecturas de Software

Duración sugerida: 3 horas

Nivel: Intermedio

Modalidad: Individual

Tecnologías: Spring Boot, Docker Compose, Grafana, Prometheus, Loki, OpenTelemetry

1. Propósito del laboratorio

En los sistemas distribuidos modernos no basta con saber si una aplicación está encendida o apagada. Un sistema puede estar respondiendo, pero hacerlo lentamente; puede estar generando errores solo en ciertos endpoints; puede tener consumidores atrasados; puede estar saturando memoria; o puede estar fallando únicamente cuando interactúa con otro servicio.

Por esta razón aparece el concepto de **observabilidad**.

La observabilidad permite comprender el comportamiento interno de un sistema a partir de las señales que produce durante su ejecución. Estas señales permiten diagnosticar problemas, analizar rendimiento, detectar errores, entender flujos distribuidos y tomar decisiones técnicas basadas en evidencia.

Este laboratorio propone una solución de observabilidad para una aplicación Spring Boot utilizando:

- **Prometheus** para recolectar métricas.
- **Grafana** para visualizar dashboards y configurar alertas.
- **Loki** para centralizar logs.
- **OpenTelemetry** para introducir el concepto de trazabilidad.
- **Docker Compose** para ejecutar el entorno de laboratorio.

Grafana permite consultar, visualizar y alertar usando diferentes fuentes de datos; Prometheus es una fuente común para métricas; Spring Boot Actuator expone métricas mediante Micrometer, incluyendo integración con Prometheus; Loki permite construir una pila de logs; y OpenTelemetry proporciona APIs, SDKs y herramientas para capturar telemetría como métricas, logs y trazas.

2. Resultados de aprendizaje

Al finalizar el laboratorio, el estudiante estará en capacidad de:

- Diferenciar monitoreo y observabilidad.
 - Explicar las señales principales de observabilidad: métricas, logs y trazas.
 - Instrumentar una aplicación Spring Boot para exponer métricas.
 - Configurar Prometheus para recolectar métricas de una aplicación.
 - Configurar Grafana para visualizar información del sistema.
 - Consultar logs centralizados mediante Loki.
 - Interpretar errores, latencia, disponibilidad y consumo de recursos.
 - Diseñar un dashboard técnico para analizar el estado de un microservicio.
 - Proponer alertas básicas para escenarios de producción.
-

3. Conceptos base

3.1 Monitoreo

El monitoreo responde principalmente preguntas como:

- ¿El servicio está disponible?
- ¿Cuánta CPU está usando?
- ¿Cuánta memoria consume?
- ¿Cuántas solicitudes recibe?
- ¿Cuántos errores se presentan?

El monitoreo suele trabajar con indicadores previamente definidos.

Ejemplo:

- Alerta si el servicio no responde.
 - Alerta si la memoria supera el 80%.
 - Alerta si los errores HTTP 500 superan cierto umbral.
-

3.2 Observabilidad

La observabilidad va más allá del monitoreo. Permite investigar comportamientos que no necesariamente fueron previstos.

Responde preguntas como:

- ¿Por qué aumentó la latencia?
- ¿Qué endpoint está fallando?
- ¿Qué servicio generó el error original?
- ¿Qué ocurrió antes de que el sistema fallara?
- ¿Qué flujo siguió una solicitud?
- ¿Qué componente está degradando el rendimiento?

En una arquitectura distribuida, estas preguntas son esenciales.

3.3 Las tres señales principales

Una solución de observabilidad suele trabajar con tres señales:

Métricas
Logs
Trazas

Métricas

Son valores numéricos recolectados en el tiempo.

Ejemplos:

Número de solicitudes HTTP.
Tiempo promedio de respuesta.
Porcentaje de errores.
Uso de memoria.
Uso de CPU.
Cantidad de hilos.
Solicitudes por segundo.

Logs

Son registros generados por una aplicación durante su ejecución.

Ejemplos:

Pedido creado correctamente.
Error al consultar inventario.
Solicitud recibida en /orders.
Timeout al llamar al servicio de pagos.

Trazas

Permiten seguir el recorrido de una solicitud a través de varios servicios.

Ejemplo:

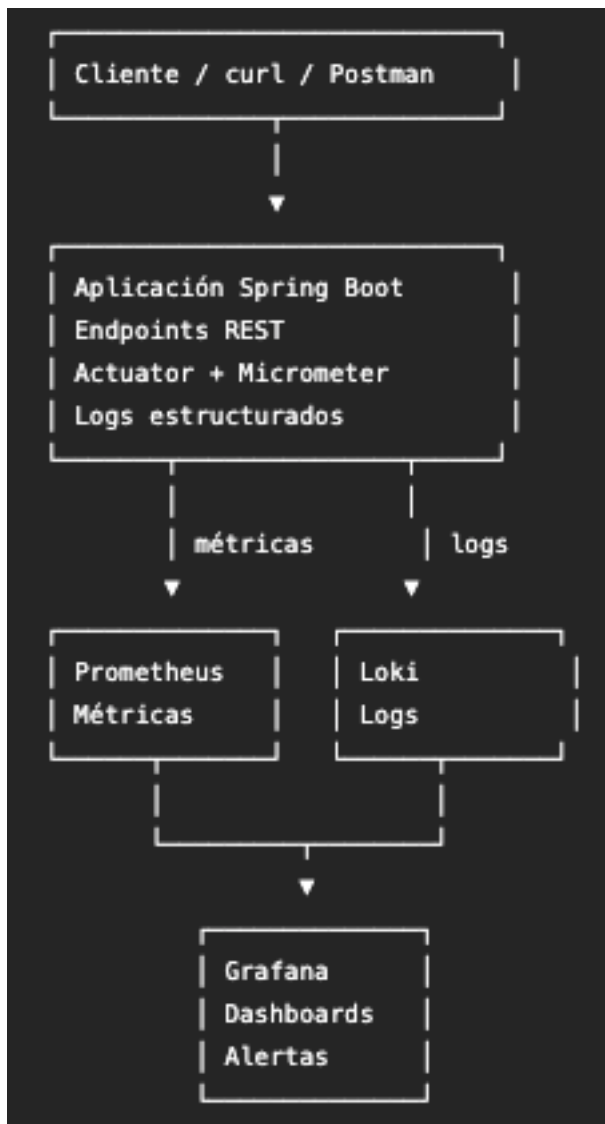
Cliente
↓
API Gateway

↓
Order Service
↓
Payment Service
↓
Inventory Service

En una arquitectura de microservicios, una traza ayuda a identificar en qué servicio ocurrió la mayor latencia o el error.

4. Arquitectura del laboratorio

La arquitectura propuesta será la siguiente:



La aplicación Spring Boot expondrá métricas en el endpoint de Actuator compatible con Prometheus. Spring Boot documenta que el endpoint prometheus entrega métricas en el formato requerido para ser recolectadas por un servidor Prometheus.

5. Requisitos previos

El estudiante debe tener instalado:

Docker
Docker Compose
Java 17 o superior
Maven 3.8 o superior
Postman, Insomnia o curl
Editor de código

Verifique las versiones:

```
docker --version
docker compose version
java --version
mvn --version
```

6. Estructura del proyecto

Cree una carpeta llamada:

arsw-observability-lab

Dentro de ella se recomienda esta estructura:

```
arsw-observability-lab
├── docker-compose.yml
├── prometheus
│   └── prometheus.yml
├── loki
│   └── loki-config.yml
├── promtail
│   └── promtail-config.yml
├── app
│   └── observability-demo
```

7. Creación de la aplicación Spring Boot

Cree un proyecto Spring Boot con las siguientes características:

Project: Maven
Language: Java
Spring Boot: 3.x
Java: 17
Group: edu.eci.arsw
Artifact: observability-demo
Name: observability-demo
Package: edu.eci.arsw.observability

Dependencias:

Spring Web
Spring Boot Actuator
Micrometer Registry Prometheus
Validation

En pom.xml, las dependencias principales serían:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>

  <dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
</dependencies>
```

Spring Boot Actuator usa Micrometer como fachada de métricas y puede integrarse con sistemas de monitoreo como Prometheus.

8. Configuración de Spring Boot Actuator

En el archivo `application.yml`, agregue:

```
server:
  port: 8081

spring:
  application:
    name: observability-demo

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      application: observability-demo

logging:
  level:
    root: INFO
    edu.eci.arsw.observability: DEBUG
```

Esta configuración permite exponer endpoints de gestión:

```
/actuator/health
/actuator/info
/actuator/metrics
/actuator/prometheus
```

El endpoint más importante para este laboratorio será:

```
http://localhost:8081/actuator/prometheus
```

9. Controlador de prueba

Cree un controlador llamado `OrderController`.

```
package edu.eci.arsw.observability.controller;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import jakarta.validation.Valid;
```

```

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;

import java.time.Duration;
import java.util.Map;
import java.util.Random;
import java.util.UUID;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private static final Logger logger = LoggerFactory.getLogger(OrderController.class);

    private final Counter orderCreatedCounter;
    private final Counter orderFailedCounter;
    private final Random random = new Random();

    public OrderController(MeterRegistry meterRegistry) {
        this.orderCreatedCounter = Counter.builder("orders_created_total")
            .description("Total de pedidos creados correctamente")
            .register(meterRegistry);

        this.orderFailedCounter = Counter.builder("orders_failed_total")
            .description("Total de pedidos fallidos")
            .register(meterRegistry);
    }

    @PostMapping
    public Map<String, Object> createOrder(@Valid @RequestBody CreateOrderRequest request) {
        logger.info("Solicitud recibida para crear pedido. customerId={}, total={}",
            request.customerId(), request.total());

        String orderId = "ORD-" + UUID.randomUUID();

        orderCreatedCounter.increment();

        logger.info("Pedido creado correctamente. orderId={}", orderId);

        return Map.of(
            "orderId", orderId,
            "customerId", request.customerId(),
            "total", request.total(),
            "status", "CREATED"
        );
    }
}

```



```

@GetMapping("/{id}")
public Map<String, Object> getOrder(@PathVariable String id) {
    logger.debug("Consultando pedido con id={}", id);

    return Map.of(
        "orderId", id,
        "status", "CREATED"
    );
}

@GetMapping("/simulate-latency")
public Map<String, Object> simulateLatency() throws InterruptedException {
    int delay = 500 + random.nextInt(2500);

    logger.warn("Simulando latencia artificial de {} ms", delay);

    Thread.sleep(Duration.ofMillis(delay));

    return Map.of(
        "message", "Respuesta con latencia simulada",
        "delayMs", delay
    );
}

@GetMapping("/simulate-error")
public Map<String, Object> simulateError() {
    logger.error("Error simulado en el servicio de pedidos");

    orderFailedCounter.increment();

    throw new IllegalStateException("Error simulado para análisis de observabilidad");
}

public record CreateOrderRequest(
    @NotBlank String customerId,
    @Min(1) double total
){
}

```

10. Manejador global de errores

Cree la clase `GlobalExceptionHandler`.

```
package edu.eci.arsw.observability.controller;
```

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public Map<String, Object> handleException(Exception ex) {
        logger.error("Excepción controlada por el manejador global", ex);

        return Map.of(
            "timestamp", Instant.now().toString(),
            "status", 500,
            "error", "Internal Server Error",
            "message", ex.getMessage()
        );
    }
}

```

Este manejador permite que los errores queden registrados en logs y que el cliente reciba una respuesta controlada.

11. Prueba inicial de la aplicación

Ejecute la aplicación:

```
mvn spring-boot:run
```

Pruebe el estado del servicio:

```
curl http://localhost:8081/actuator/health
```

Debe obtener una respuesta similar:

```
{
  "status": "UP"
}
```

Pruebe el endpoint de métricas Prometheus:

```
curl http://localhost:8081/actuator/prometheus
```

Debe observar métricas en texto plano.

Busque métricas como:

```
http_server_requests_seconds_count
jvm_memory_used_bytes
process_cpu_usage
orders_created_total
orders_failed_total
```

12. Generación de tráfico

Ejecute varias solicitudes:

```
curl -X POST http://localhost:8081/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUS-01","total":120000}'

curl http://localhost:8081/orders/ORD-1001

curl http://localhost:8081/orders/simulate-latency

curl http://localhost:8081/orders/simulate-error
```

Repita estas solicitudes varias veces para generar datos observables.

13. Configuración de Prometheus

Cree la carpeta:

```
mkdir prometheus
```

Dentro cree el archivo:

```
prometheus/prometheus.yml
```

Contenido:

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'observability-demo'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['host.docker.internal:8081']
```

En Linux, si `host.docker.internal` no funciona, puede ser necesario usar la IP del host o configurar la red de Docker.

Prometheus recolectará las métricas de la aplicación cada 5 segundos.

14. Configuración de Loki

Cree la carpeta:

```
mkdir loki
```

Archivo:

```
loki/loki-config.yml
```

Contenido básico para laboratorio:

```
auth_enabled: false
```

```
server:
```

```
  http_listen_port: 3100
```

```
common:
```

```
  path_prefix: /loki
```

```
  storage:
```

```
    filesystem:
```

```
      chunks_directory: /loki/chunks
```

```
      rules_directory: /loki/rules
```

```
  replication_factor: 1
```

```
  ring:
```

```
    kvstore:
```

```
      store: inmemory
```

```
schema_config:
```

```
  configs:
```

```
    - from: 2024-01-01
```

```
      store: tsdb
```

```
      object_store: filesystem
```

```
      schema: v13
```

```
      index:
```

```
        prefix: index_
```

```
        period: 24h
```

Loki está diseñado como una pila de logs de bajo costo operacional basada en índices pequeños y chunks comprimidos.

15. Configuración de Promtail

Promtail enviará logs hacia Loki.

Cree la carpeta:

```
mkdir promtail
```

Archivo:

```
promtail/promtail-config.yml
```

Contenido:

```
server:
```

```
  http_listen_port: 9080
```

```
  grpc_listen_port: 0
```

```
positions:
```

```
  filename: /tmp/positions.yml
```

```
clients:
```

```
  - url: http://loki:3100/loki/api/v1/push
```

```
scrape_configs:
```

```
  - job_name: docker-logs
```

```
    static_configs:
```

```
      - targets:
```

```
        - localhost
```

```
      labels:
```

```
        job: docker
```

```
        __path__: /var/log/*.log
```

Para un laboratorio simple, también puede trabajarse con logs de contenedores si la aplicación corre dentro de Docker. En una versión más avanzada se puede configurar Promtail para leer los logs específicos de la aplicación.

16. Docker Compose del entorno

Cree el archivo:

```
docker-compose.yml
```

Contenido:

```
services:
```

```
  prometheus:
```

```
    image: prom/prometheus:latest
```

```
    container_name: arsw-prometheus
```

```
ports:
  - "9090:9090"
volumes:
  - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
```

```
grafana:
  image: grafana/grafana:latest
  container_name: arsw-grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
  depends_on:
    - prometheus
    - loki
```

```
loki:
  image: grafana/loki:latest
  container_name: arsw-loki
  ports:
    - "3100:3100"
  command: -config.file=/etc/loki/local-config.yml
  volumes:
    - ./loki/loki-config.yml:/etc/loki/local-config.yml
```

```
promtail:
  image: grafana/promtail:latest
  container_name: arsw-promtail
  volumes:
    - ./promtail/promtail-config.yml:/etc/promtail/config.yml
    - /var/log:/var/log
  command: -config.file=/etc/promtail/config.yml
  depends_on:
    - loki
```

Levante el entorno:

```
docker compose up -d
```

Verifique:

```
docker ps
```

Debe observar:

```
arsw-prometheus
arsw-grafana
arsw-loki
arsw-promtail
```

17. Verificación de Prometheus

Abra:

`http://localhost:9090`

Ingresa a:

Status → Targets

Debe aparecer el target:

observability-demo

Estado esperado:

UP

Si aparece DOWN, revise:

La aplicación Spring Boot está encendida.

El puerto 8081 está disponible.

La ruta `/actuador/prometheus` responde.

La dirección `host.docker.internal` funciona en su sistema operativo.

18. Consultas básicas en Prometheus

En Prometheus, pruebe consultas como:

`up`

`http_server_requests_seconds_count`

`jvm_memory_used_bytes`

`orders_created_total`

`orders_failed_total`

Estas consultas permiten validar que Prometheus está recolectando métricas de la aplicación.

19. Ingreso a Grafana

Abra:

`http://localhost:3000`

Credenciales:

Usuario: admin

Contraseña: admin

Grafana solicitará cambiar la contraseña. Para laboratorio puede omitir el cambio o definir una contraseña sencilla.

20. Configurar Prometheus como fuente de datos

En Grafana:

Connections

Data sources

Add data source

Prometheus

Configure:

URL: `http://prometheus:9090`

Guarde y pruebe la conexión.

Grafana documenta Prometheus como fuente de datos para consultar y visualizar métricas recolectadas por Prometheus.

21. Configurar Loki como fuente de datos

En Grafana:

Connections

Data sources

Add data source

Loki

Configure:

URL: `http://loki:3100`

Guarde y pruebe la conexión.

22. Construcción del dashboard

Cree un dashboard llamado:

Agregue los siguientes paneles.

Panel 1: estado del servicio

Consulta PromQL:

```
up{job="observability-demo"}
```

Tipo de visualización sugerido:

Stat

Interpretación:

1 significa que Prometheus puede consultar la aplicación.

0 significa que la aplicación no está disponible para Prometheus.

Panel 2: solicitudes HTTP por endpoint

Consulta:

```
sum by (uri, method, status) (rate(http_server_requests_seconds_count[1m]))
```

Tipo:

Time series

Interpretación:

Permite ver cuántas solicitudes por segundo recibe cada endpoint.

Panel 3: latencia promedio

Consulta:

```
sum(rate(http_server_requests_seconds_sum[1m]))  
/  
sum(rate(http_server_requests_seconds_count[1m]))
```

Tipo:

Time series

Interpretación:

Permite identificar si el tiempo promedio de respuesta aumenta.

Panel 4: errores HTTP 500

Consulta:

```
sum(rate(http_server_requests_seconds_count{status="500"}[1m]))
```

Tipo:

Time series

Interpretación:

Permite observar la tasa de errores internos.

Panel 5: pedidos creados

Consulta:

```
orders_created_total
```

Tipo:

Stat

Interpretación:

Muestra la cantidad total de pedidos creados correctamente.

Panel 6: pedidos fallidos

Consulta:

```
orders_failed_total
```

Tipo:

Stat

Interpretación:

Muestra la cantidad de errores simulados en el flujo de pedidos.

Panel 7: memoria usada por JVM

Consulta:

```
sum(jvm_memory_used_bytes{application="observability-demo"})
```

Tipo:

Time series

Interpretación:

Permite observar el consumo de memoria de la aplicación.

Panel 8: uso de CPU del proceso

Consulta:

```
process_cpu_usage{application="observability-demo"}
```

Tipo:

Time series

Interpretación:

Permite observar la carga aproximada del proceso Java.

23. Exploración de logs con Loki

En Grafana, ingrese a:

Explore

Seleccione la fuente de datos:

Loki

Pruebe consultas como:

```
{job="docker"}
```

Para buscar errores:

```
{job="docker"} |= "ERROR"
```

Para buscar logs relacionados con pedidos:

```
{job="docker"} |= "Pedido"
```

Para buscar latencia:

```
{job="docker"} |= "latencia"
```

24. Simulación de incidentes

El estudiante debe generar tres incidentes controlados.

Incidente 1: aumento de errores

Ejecute varias veces:

```
curl http://localhost:8081/orders/simulate-error
```

Observe:

Panel de errores HTTP 500.
Contador `orders_failed_total`.
Logs con nivel ERROR.

Preguntas:

- ¿Qué endpoint generó errores?
- ¿Qué métrica permitió detectarlo?
- ¿Qué log permitió explicar el error?

Incidente 2: aumento de latencia

Ejecute varias veces:

```
curl http://localhost:8081/orders/simulate-latency
```

Observe:

Panel de latencia promedio.
Logs con advertencias.
Tiempo de respuesta del cliente.

Preguntas:

- ¿Qué métrica cambió?
- ¿Qué endpoint parece más lento?
- ¿Qué log confirma la latencia artificial?

Incidente 3: creación de pedidos

Ejecute varias veces:

```
curl -X POST http://localhost:8081/orders \  
-H "Content-Type: application/json" \  
-d '{"customerId":"CUS-01","total":120000}'
```

Observe:

Panel de solicitudes HTTP.
Panel de pedidos creados.
Logs de creación de pedidos.

Preguntas:

¿Qué métrica evidencia actividad de negocio?
¿Qué log permite rastrear un pedido específico?
¿Qué información adicional agregaría para mejorar trazabilidad?

25. Introducción a OpenTelemetry

OpenTelemetry es un estándar abierto para generar y exportar telemetría. Su objetivo es evitar que las aplicaciones dependan de un único proveedor de observabilidad.

Permite trabajar con:

Métricas
Logs
Trazas

En esta guía, OpenTelemetry se introduce conceptualmente. En una versión extendida del laboratorio, se puede agregar el OpenTelemetry Collector y un backend de trazas para visualizar recorridos completos entre servicios.

OpenTelemetry se define como un framework de observabilidad para software cloud native, con APIs, bibliotecas, agentes y servicios de collector para capturar telemetría.

26. ¿Qué aportan las trazas?

Una métrica puede decir:

La latencia aumentó.

Un log puede decir:

El endpoint /orders/simulate-latency tardó 2400 ms.

Una traza puede mostrar:

La solicitud inició en API Gateway.
Luego pasó por Order Service.
Después llamó a Payment Service.
El mayor tiempo ocurrió en Payment Service.

En sistemas con muchos microservicios, las trazas son fundamentales para entender el recorrido completo de una solicitud.

27. Diseño de alertas

En Grafana se pueden crear alertas sobre consultas.

Para este laboratorio, proponga las siguientes alertas:

Alerta 1: servicio caído

Condición:

```
up{job="observability-demo"} == 0
```

Interpretación:

Prometheus no puede consultar la aplicación.

Alerta 2: errores HTTP 500

Condición:

```
sum(rate(http_server_requests_seconds_count{status="500"}[1m])) > 0
```

Interpretación:

La aplicación está generando errores internos.

Alerta 3: latencia elevada

Condición:

```
(  
  sum(rate(http_server_requests_seconds_sum[1m]))  
  /  
  sum(rate(http_server_requests_seconds_count[1m]))  
) > 1
```

Interpretación:

La latencia promedio supera un segundo.

28. Actividad 1: diagnóstico de observabilidad

Después de ejecutar los incidentes simulados, complete:

Incidente observado:

Métrica afectada:

Logs relacionados:

Endpoint involucrado:

Posible causa:

Impacto para el usuario:

Acción correctiva propuesta:

Alerta que debería existir:

29. Actividad 2: diseño de dashboard

Diseñe un dashboard para un microservicio de pedidos.

Debe incluir al menos:

Disponibilidad del servicio.

Solicitudes por segundo.

Latencia promedio.

Errores HTTP 500.

Pedidos creados.

Pedidos fallidos.

Uso de memoria.

Uso de CPU.

Logs de error.

Para cada panel indique:

Nombre del panel:

Fuente de datos:

Consulta:

Qué permite analizar:

Decisión técnica que soporta:

30. Actividad 3: observabilidad en arquitectura

Una plataforma de comercio electrónico cuenta con estos servicios:

order-service
payment-service
inventory-service
notification-service
shipping-service

Proponga una estrategia de observabilidad.

Debe incluir:

Métricas importantes por servicio.
Logs que deberían registrarse.
Trazas necesarias.
Alertas recomendadas.
Dashboards requeridos.
Indicadores de negocio.
Indicadores técnicos.

Responda:

¿Qué señales permitirían detectar un problema en pagos?
¿Qué señales permitirían detectar lentitud en inventario?
¿Qué señales permitirían saber si las notificaciones fallan?
¿Qué métrica usaría para medir disponibilidad?
¿Qué métrica usaría para medir experiencia del usuario?

31. Reto final

El estudiante debe entregar una propuesta de observabilidad para la el proyecto de curso. Inicialmente configurado para 2 microservicios desarrollados.

La entrega debe incluir:

1. Descripción de la arquitectura de observabilidad.
 2. Captura del estado de Prometheus Targets.
 3. Captura del dashboard de Grafana.
 4. Evidencia de métricas HTTP.
 5. Evidencia de métricas personalizadas.
 6. Evidencia de logs consultados en Loki.
 7. Análisis de un incidente simulado.
 8. Propuesta de tres alertas.
 9. Recomendaciones para llevar la solución a producción.
-

32. Rúbrica de evaluación

Criterio	Descripción	Peso
Entorno funcional	Levanta correctamente Prometheus, Grafana y Loki.	20%
Instrumentación	La aplicación expone métricas técnicas y de negocio.	20%
Dashboard	El dashboard permite analizar disponibilidad, latencia, errores y recursos.	20%
Logs	El estudiante consulta logs relevantes para explicar incidentes.	15%
Análisis técnico	Relaciona métricas, logs y causas probables de incidentes.	15%
Propuesta de alertas	Define alertas útiles y accionables.	10%

33. Cierre del laboratorio

Este laboratorio permitió construir una solución práctica de observabilidad para un microservicio Spring Boot. El estudiante configuró métricas con Actuator y Micrometer, recolectó información con Prometheus, visualizó datos en Grafana y exploró logs con Loki.

La observabilidad no consiste únicamente en crear dashboards atractivos. Su valor está en permitir que un equipo pueda responder preguntas técnicas importantes:

¿El sistema está disponible?
¿Qué endpoint está fallando?
Dónde aumentó la latencia?
Qué error aparece en los logs?
Qué métrica confirma el problema?
Qué alerta debería activarse?
Qué decisión técnica debe tomarse?

En arquitecturas distribuidas, una buena estrategia de observabilidad es tan importante como el diseño de servicios, bases de datos, eventos o APIs. Sin observabilidad, los sistemas fallan en silencio o se vuelven difíciles de diagnosticar.